

Семинар №4

Итераторы.

Продолжение продолжения
(Потоки и контейнеры)

stringstream

- Позволяет работать со строкой как с потоком
- Можно использовать для преобразования типа
- А также для форматирования строк
- `istringstream` – чтение из строки как из потока
- `ostringstream` – запись в строку как в поток
- `stringstream` – объединяет функционал `istringstream/ostringstream`

Работа с файлами

- Классы **std::ifstream** / **std::ofstream** / **std::fstream** для чтения из файла / записи в файл / чтения и записи
- В каких режимах можно открыть файл (можно объединять "|"):
 - **std::ios::in** – на чтение
 - **std::ios::out** – на запись
 - **std::ios::ate** – передвинуть каретку в конец файла
 - **std::ios::app** – (append) запись в конец файла вне зависимости от каретки
 - **std::ios::binary** – бинарный формат
 - **std::ios::trunc** – отчистить содержимое файла при открытии
 - **std::ios::noreplace** – если файл уже существует, бросается исключение
- **seekp/tellp** & **seekg/tellg** – изменение/чтение позиции на запись & чтение

Итераторы по потокам

- Класс `istream_iterator` / `ostream_iterator`
- Обертка над потоком, позволяющая получить итератор на него
- Являются `input/output`, но не `forward` итераторами

std::string_view

- Позволяют получить “взгляд” на контейнеры, но не дают владения им (!). Надо следить, чтобы “вьюшка” не пережила сам контейнер
- Лучше принимать в функции string_view, чем const string& (конверсия из C-style строк и литералов ведёт к лишним копированиям)
- Бесплатные подстроки (в отличие от string.substr)
- Через std::string_view нельзя менять исходную строку

std::span

- Легковесный “взгляд” на контейнер с последовательными элементами
- Можно работать с C-style массивами, векторами, std::array, диапазонами из них
- Избегает лишних копий
- Бесплатное создание частей последовательности (.subspan)
- Через std::span можно менять исходную последовательность
- Надо следить, что std::span не переживает последовательность

std::optional

- Значение либо есть, либо его нет
- Позволяет избегать -1 и прочих магических констант
- Позволяет отложить создание объекта на потом
- Максимально упаковывает флаг (в паддинг, если возможно)
- При попытке доступа к объекту, когда его нет, вылетает исключение
- std::nullopt – отсутствие объекта
- Есть каст к bool
- Объект лежит на стеке, никаких внутренних динамических аллокаций
- value_or(default) – значение или дефолтное значение

std::expected

- Похож на std::optional, но может иметь несколько “ошибочных” значений
- Альтернатива исключениям
- При попытке обращения к несуществующему значению выбрасывается исключение **std::bad_expected_access**
- Monadic interface: and_then, transform, or_else
- value_or(default) – значение или дефолтное значение
- В качестве ошибки можно использовать std::error_code или использовать свой класс / enum
- Хранит union из двух типов, пытается уместить флаг в паддинг

Что почитать

- [Fstream](#)